WEEK 1 · DAY 3 · OVERVIEW

Random Forests



Created by **Jeff Lenamon**

ABOUT 45 MIN WITH EXERCISES / 11 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

Day 2's bag of trees, with leaves chosen out of bag, reached a validation ROC AUC of 0.7837, past Course 4's pruned tree.

Yet all its members asked **`PAY_0`** first, and Day 2 closed by asking for members that differ more without getting weaker.

A random forest is the standard next step: it changes what each tree may ask at every split.

The desk wants to know what that changes, and which of the forest's settings matter.

Settings are chosen out of bag inside the 18,000 training rows; the validation rows only report.

The accounts are real, from Taiwan in 2005, so they describe consumer credit there and then, not US credit today.

The four person columns are never features, because a model that used them would sort people by them.

OBJECTIVES

Fit a random forest with scikit-learn and read the question each of its trees asks first.

Measure how strong a forest's trees are and how much they agree, sweep the column count and the leaf size, and map every forest by its trees.

Choose the settings out of bag, report the chosen forest on validation rows, watch its score settle as trees are added, and fit extremely randomized trees beside it.

ABOUT 45 MIN WITH EXERCISES / 11 SECTIONS / 4 EXERCISES

Your Route Through Day 3

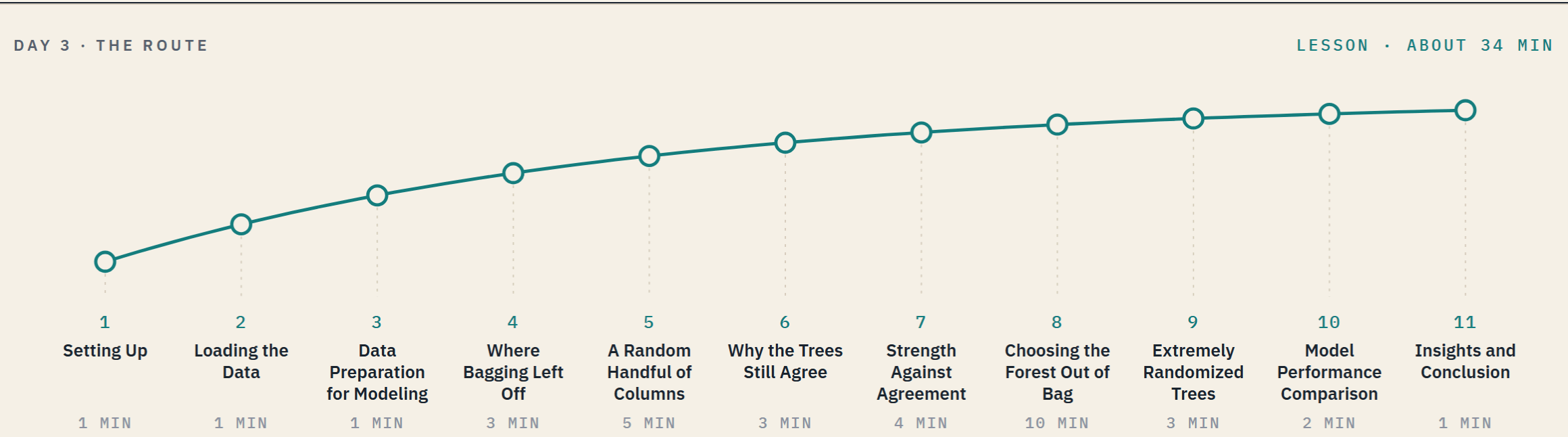


FIGURE 1 Eleven sections, in order, with about how long each takes.

Time. About 34 minutes for the lesson, then about 12 for the four exercises.

Fix the seed and load the libraries.

Setting Up

THE SETUP

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
DATA = "../../../data/"
if not Path(DATA + "benchmark.csv").exists():
    DATA = "https://raw.githubusercontent.com/lenamonj/python-fixed-income/main/data/"
```

THE LIBRARIES

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier, RandomForestClassifier, ExtraTreesClassifier
```

Loading the Data

THE FILE'S COLUMNS

- **LIMIT_BAL**: credit limit, NT dollars
- **PAY_0**, **PAY_2** to **PAY_6**: repayment status, September back to April 2005, where -1 is paid on time, 1 to 9 is months late, and -2 and 0 are undocumented (used as numbers, a modelling choice, as in Course 4)
- **BILL_AMT1** to **BILL_AMT6**: the bill, NT dollars, September back to April
- **PAY_AMT1** to **PAY_AMT6**: the amount paid, NT dollars, September back to April
- **default payment next month**: 1 if the account defaulted the following month, **the target**
- **ID**, **SEX**, **EDUCATION**, **MARRIAGE**, **AGE**: never used

READING THE CARD FILE

```
card = pd.read_csv(DATA + "credit_card_default.csv")
TARGET = "default payment next month"
print(len(FEATURES), "features")
```

```
19 features
```

THE SHAPE

```
print(card.shape)
```

```
(30000, 25)
```

Rebuild Course 4's training, validation and test rows.

Data Preparation for Modeling

COURSE 4'S SPLIT

```
X_rest, X_test, y_rest, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=SEED)
X_train, X_valid, y_train, y_valid = train_test_split(X_rest, y_rest, test_size=0.25, stratify=y_rest, random_state=SEED)
print("Training:", len(X_train), " Validation:", len(X_valid), " Test:", len(X_test))
```

```
Training: 18000  Validation: 6000  Test: 6000
```

WHO DOES WHAT

The same accounts as Course 4 and Days 1 and 2.

Every score below is on the validation rows; Course 5 chooses nothing on them and never scores the test rows.

Training rows fit and choose, validation rows report, test rows stay closed.

Where Bagging Left Off

STRENGTH AND AGREEMENT

```
def tree_outputs(model):
    return np.column_stack([tree.predict_proba(X_valid.to_numpy())[:, 1] for tree in model.estimators_])
def describe_forest(model):
    outputs = tree_outputs(model)
    pairs = np.triu_indices(outputs.shape[1], k=1)
    correlations = np.corrcoef(outputs.T)[pairs]
    strength = np.mean([roc_auc_score(y_valid, outputs[:, j]) for j in range(outputs.shape[1])])
    return correlations, strength
```

THE BAG'S MEMBERS

```
Pairs: 19900  Mean correlation: 0.2567
Strength: 0.6161
```

THE FIRST QUESTION

```
PAY_0    200
Name: count, dtype: int64
```

Every member opens with the same question, and the members agree at 0.2567. A forest makes them ask different ones.

A Random Handful of Columns

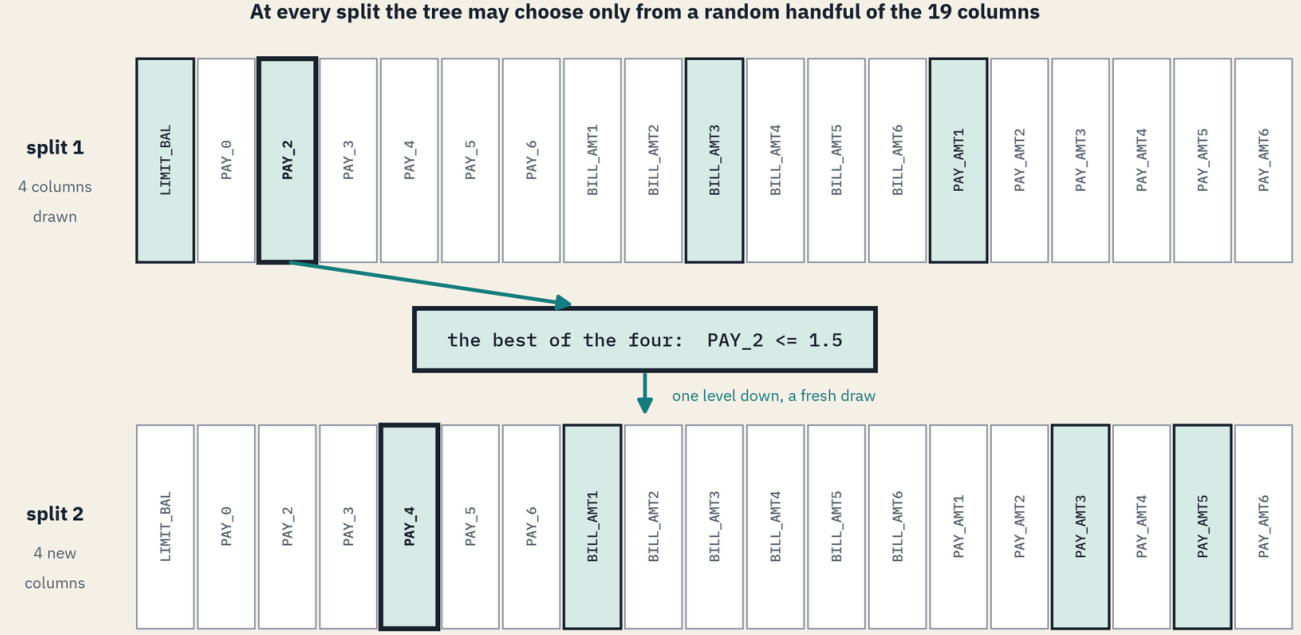


FIGURE 2 One split offers 4 of the 19 columns at random and takes the best of them; the next split draws 4 afresh.

THE ONE CHANGE

A **random forest** is bagging with one change: at every split, a tree may choose only from a random handful of the columns, drawn afresh at that split.

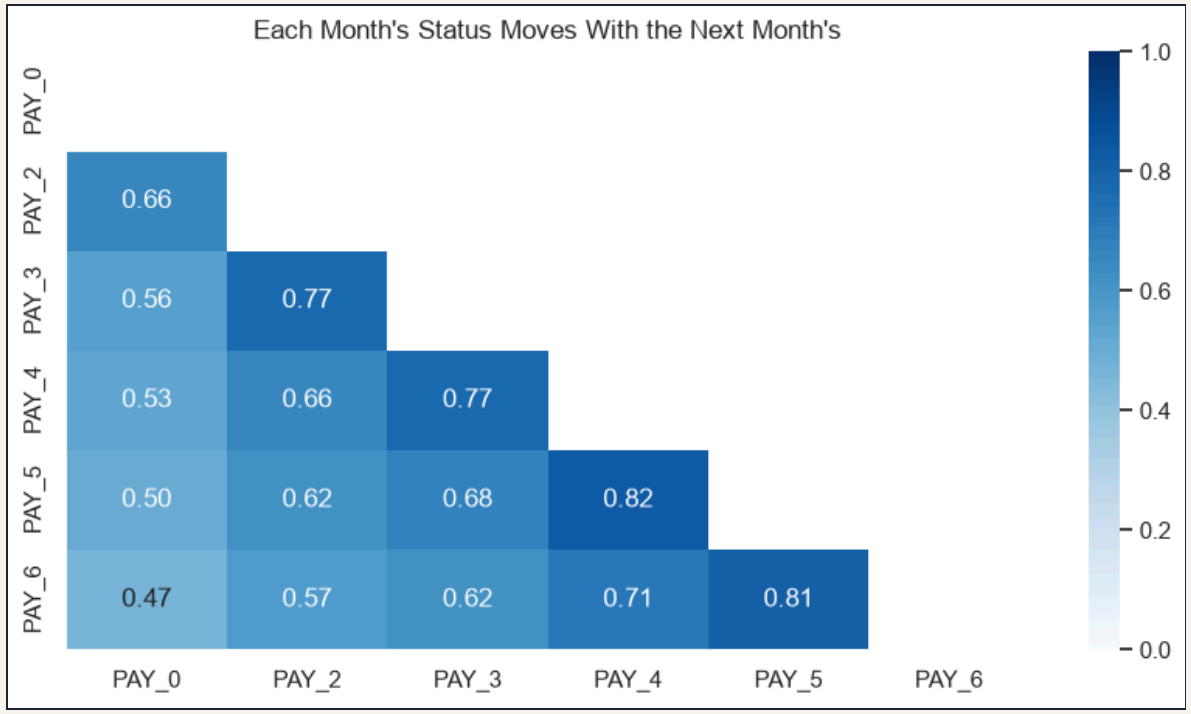
`max_features` sets the handful; its default, `"sqrt"`, offers the whole part of the square root of the column count, 4 of our 19.

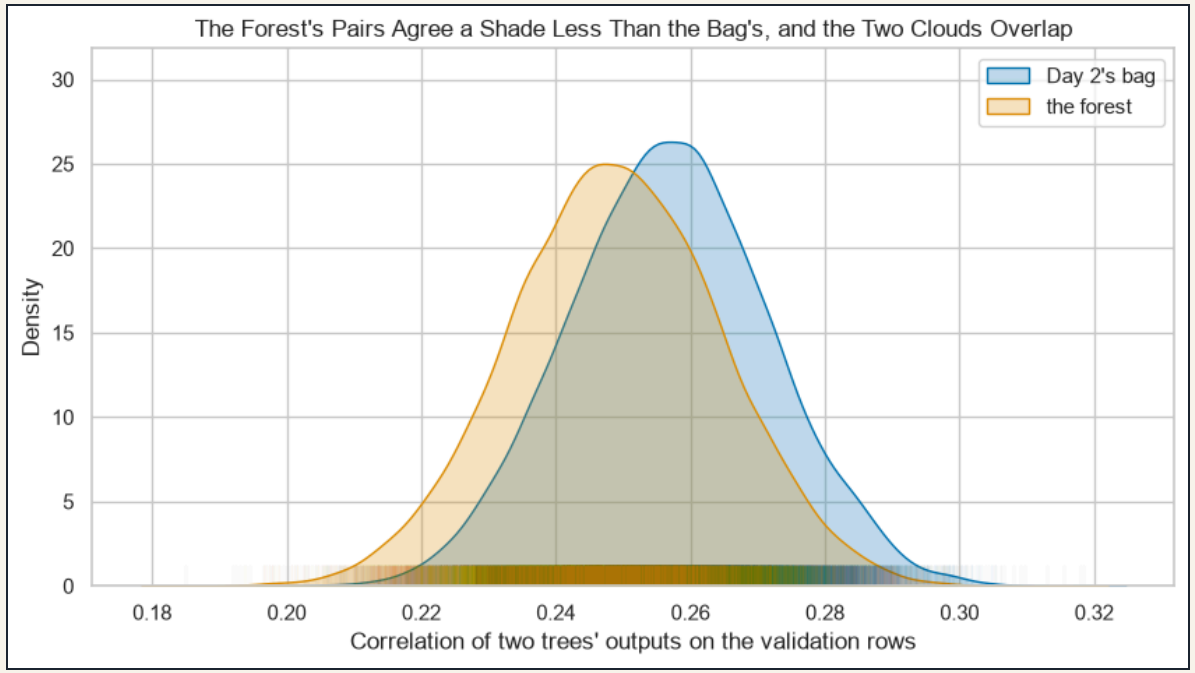
A FOREST IN ONE CALL

```
forest = RandomForestClassifier(n_estimators=200, oob_score=True, random_state=SEED, n_jobs=1).fit(X_train, y_train)
```

Compare the status columns and the trees' agreement.

Why the Trees Still Agree





The other lever is the trees themselves: stop them memorising and watch strength and agreement.

Strength Against Agreement

WHAT A LEAF SIZE IS

min_samples_leaf sets the fewest training accounts a leaf may hold, the setting Day 2 chose out of bag for its bag.

THE QUESTION

Q. Take the forest with 4 columns per split and leaves of at least 100 accounts. What is the mean correlation between its trees? Write down a number.

THIRTY FORESTS

```
rows = []
for columns in [1, 2, 4, 8, 19]:
    for leaf in [1, 3, 10, 30, 100, 300]:
        model = RandomForestClassifier(n_estimators=50, max_features=columns, min_samples_leaf=leaf, oob_score=True,
                                       random_state=SEED, n_jobs=1).fit(X_train, y_train)
        correlations, strength = describe_forest(model)
        rows.append([columns, leaf, correlations.mean(), strength, roc_auc_score(y_train, model.oob_decision_function_[:, 1])])
grid = pd.DataFrame(rows, columns=["max_features", "min_samples_leaf", "correlation", "strength", "oob_roc_auc"])
```

`max_features=19` offers every column at every split, so that column of the grid is a bag.

Choosing the Forest Out of Bag

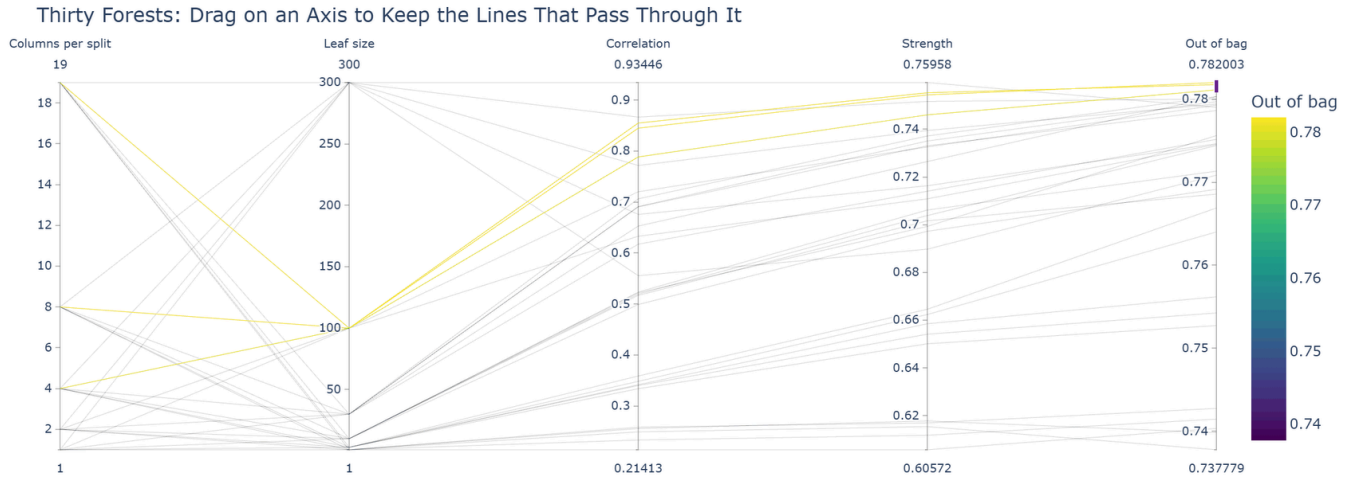


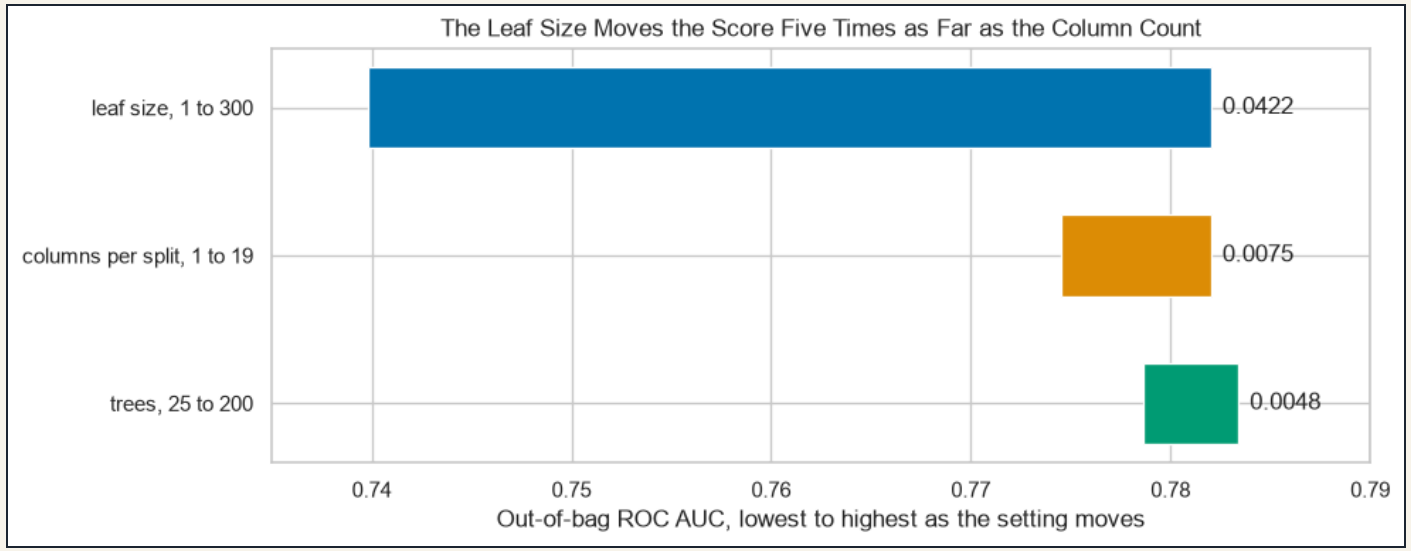
FIGURE 3 The thirty forests with the out-of-bag axis dragged to its top three: all three pass through leaf size 100.

OUT-OF-BAG SCORES

max_features	1	2	4	8	19
min_samples_leaf					
1	0.7404	0.7415	0.7378	0.7398	0.7428
3	0.7669	0.7640	0.7562	0.7543	0.7527
10	0.7756	0.7744	0.7713	0.7686	0.7691
30	0.7752	0.7803	0.7797	0.7803	0.7786
100	0.7745	0.7793	0.7811	0.7820	0.7818
300	0.7708	0.7746	0.7794	0.7800	0.7790

We take 8 columns per split and leaves of at least 100 accounts; the default 4 would do almost as well.

Which Setting Matters



HOW FAR EACH SETTING MOVES THE SCORE

	LOW	HIGH	RANGE
leaf size, 1 to 300	0.7398	0.7820	0.0422
columns per split, 1 to 19	0.7745	0.7820	0.0075
trees, 25 to 200	0.7786	0.7834	0.0048

READING THE BARS

The leaf size moves the out-of-bag score by 0.0422, the column count by 0.0075, and the trees from 25 to 200 by 0.0048.

Leaf size first, then enough trees; the column count fine-tunes. One more kind of randomness is left to try.

Extremely Randomized Trees

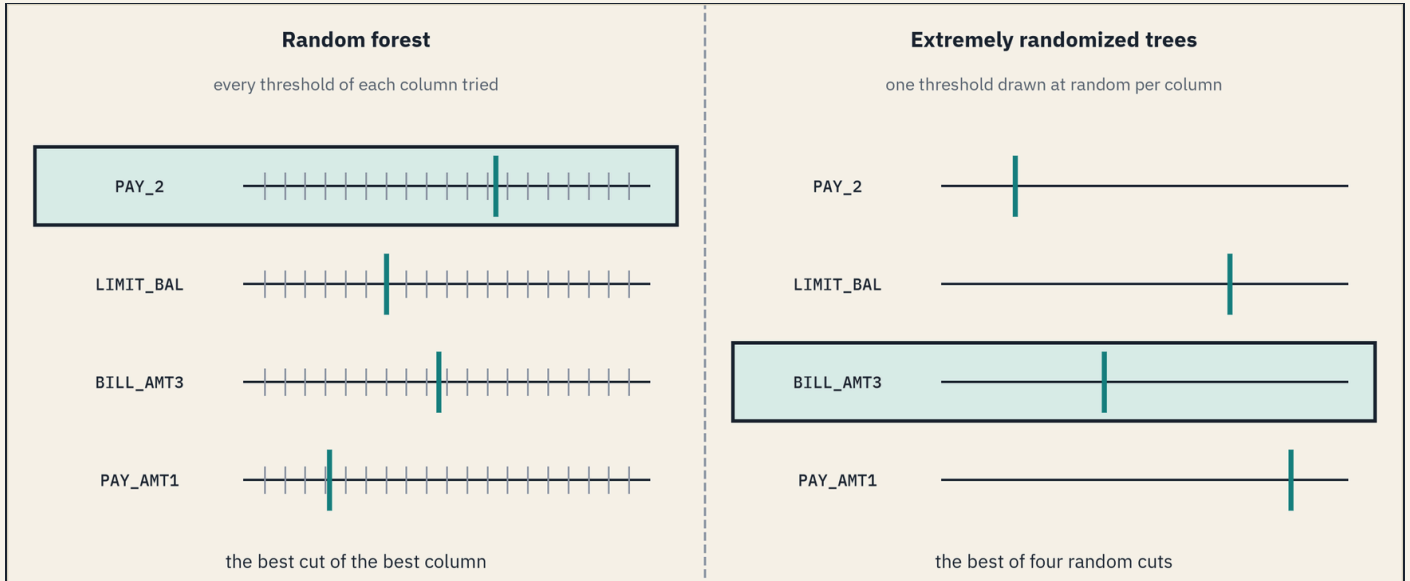


FIGURE 4 A forest tries every threshold of each offered column; extra trees draw one random threshold per column.

THE QUESTION

Q. With the chosen forest's settings, 200 trees, 8 columns and 100-account leaves, what do extremely randomized trees score on the validation rows? Write down a number.

THE IDEA

Extremely randomized trees go one step further: for each offered column they draw one threshold at random, and keep the best of those few cuts.

THE FIT, AND WHERE EACH TREE CUTS PAY_0

```
extra = ExtraTreesClassifier(n_estimators=200, max_features=8, min_samples_leaf=100, random_state=SEED, n_jobs=1).fit(X_train, y_train)
```

```
Chosen forest: {1.5: 76}
Extra trees: 29 roots on PAY_0, cut at [-1.7, -1.47, -0.95, -0.71, -0.48, -0.32, -0.26, 0.1] ...
```

Line up five models on the validation rows.

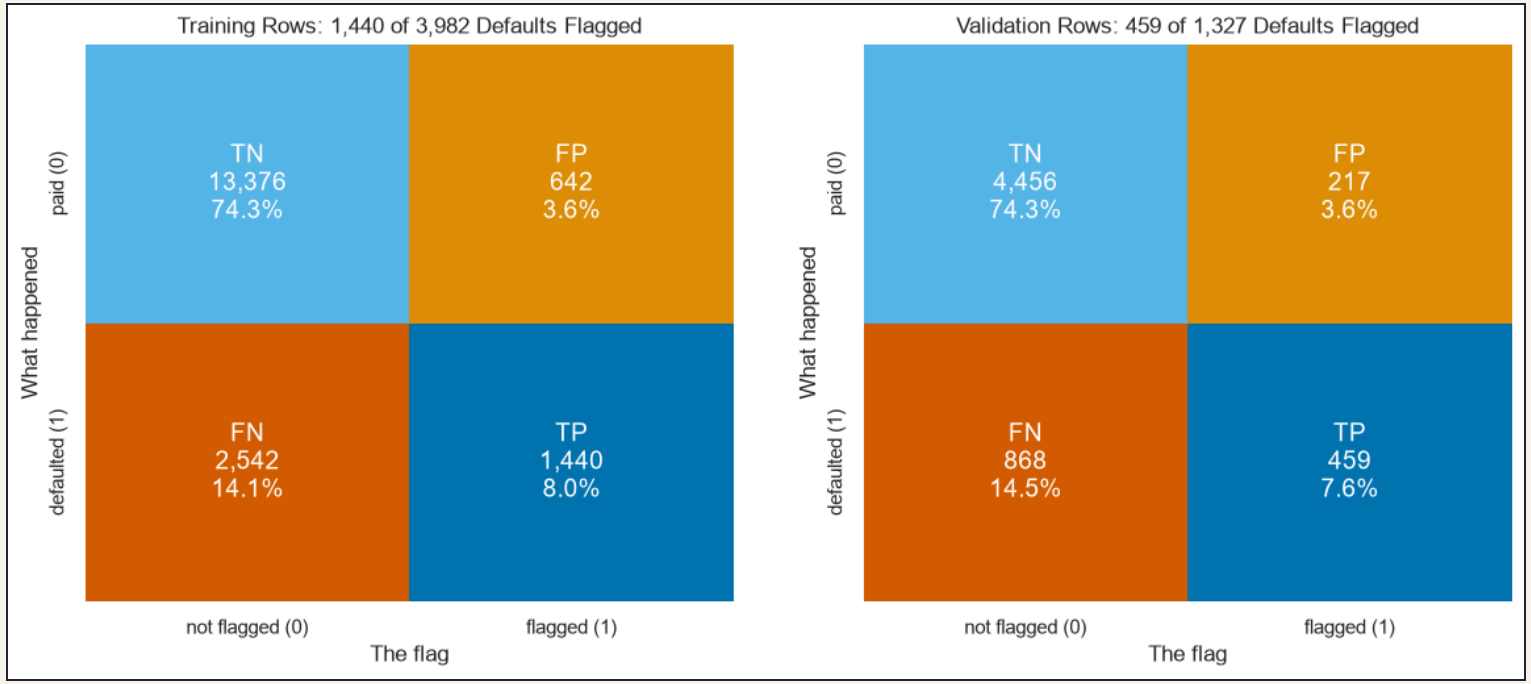
Model Performance Comparison

	ACCURACY	RECALL	PRECISION	F1
Course 4's pruned tree	0.7683	0.5840	0.4805	0.5272
Day 2's bag	0.8165	0.3760	0.6464	0.4755
chosen forest	0.8192	0.3459	0.6790	0.4583

ONE FUNCTION SCORES EVERY MODEL

```
def model_performance_classification(model, predictors, target, threshold=0.5):
    flags = (model.predict_proba(predictors)[:, 1] >= threshold).astype(int)
    return pd.DataFrame({
        "ROC AUC": [roc_auc_score(target, model.predict_proba(predictors)[:, 1])],
        "Accuracy": [accuracy_score(target, flags)],
        "Recall": [recall_score(target, flags, zero_division=0)],
        "Precision": [precision_score(target, flags, zero_division=0)],
        "F1": [f1_score(target, flags, zero_division=0)],
    })
```

Insights and Conclusion



THE CHOSEN FOREST'S TREES

```
Leaves: 80 to 92
Depth: 12 to 19
```

AGAINST A FULL TREE

Its trees have 80 to 92 leaves and are 12 to 19 questions deep, against thousands of leaves for a full tree.

Day 4 asks which columns the chosen forest leans on, and how far to trust the answer.

You Can Now

- Fit a random forest with **`RandomForestClassifier`** and read the first question of each of its trees.
- Measure a forest's strength and the agreement between its trees.
- Sweep **`max_features`** and **`min_samples_leaf`**, and draw every forest on one map.
- Choose a forest's settings out of bag and report it on validation rows.
- Watch a forest's score settle as **`n_estimators`** grows, and fit **`ExtraTreesClassifier`**.

NEXT
WEEK 1 · DAY 4

Impurity and Permutation Importance

Day 4 asks which columns the chosen forest leans on, and how far to trust it.